Eco Routes SVM

Security Review

Cantina Managed review by:
Rikard Hjort, Lead Security Researcher
Jay, Security Researcher

May 9, 2026

Contents

1 Introduction

1.1 About Cantina

Cantina is a security services marketplace that connects top security researchers and solutions with clients. Learn more at cantina.xyz

1.2 Disclaimer

Cantina Managed provides a detailed evaluation of the security posture of the code at a particular moment based on the information available at the time of the review. While Cantina Managed endeavors to identify and disclose all potential security issues, it cannot guarantee that every vulnerability will be detected or that the code will be entirely secure against all possible attacks. The assessment is conducted based on the specific commit and version of the code provided. Any subsequent modifications to the code may introduce new vulnerabilities that were absent during the initial review. Therefore, any changes made to the code require a new security review to ensure that the code remains secure. Please be advised that the Cantina Managed security review is not a replacement for continuous security measures such as penetration testing, vulnerability scanning, and regular code reviews.

1.3 Risk assessment

Severity level	Impact: High	Impact: Medium	Impact: Low
Likelihood: high	Critical	High	Medium
Likelihood: medium	High	Medium	Low
Likelihood: low	Medium	Low	Low

1.3.1 Severity Classification

The severity of security issues found during the security review is categorized based on the above table. Critical findings have a high likelihood of being exploited and must be addressed immediately. High findings are almost certain to occur, easy to perform, or not easy but highly incentivized thus must be fixed as soon as possible.

Medium findings are conditionally possible or incentivized but are still relatively likely to occur and should be addressed. Low findings are a rare combination of circumstances to exploit, or offer little to no incentive to exploit but are recommended to be addressed.

Lastly, some findings might represent objective improvements that should be addressed but do not impact the project's overall security (Gas and Informational findings).

2 Security Review Summary

Eco enables apps to unlock stablecoin liquidity from any connected chain and give users the simplest onchain experience.

From Apr 24th to Apr 30th the Cantina team conducted a review of eco-routes-svm on commit hash f08f65b5. The team identified a total of **5** issues:

Issues Found

Severity	Count	Fixed	Acknowledged
Critical Risk	0	0	0
High Risk	0	0	0
Medium Risk	0	0	0
Low Risk	0	0	0
Gas Optimizations	0	0	0
Informational	5	4	1
Total	**5**	**4**	**1**

3 Findings

3.1 Informational

3.1.1 `flash_fulfill` refunds buffer rent to caller-supplied payer instead of recorded writer

Severity: Informational

Context: flash_fulfill.rs#L203

Description: `set_flash_fulfill_intent` lets a writer publish a (route, reward) buffer at a PDA keyed only by `intent_hash`, paying the rent and recording themselves as writer. When `flash_fulfill` consumes that buffer via the `IntentHash` variant, it closes the account with `destination = payer`, never reading or checking `flash_fulfill_intent.writer`. Because `payer` is supplied by the caller and consumption isn't gated on `writer`, any solver who watches the mempool and submits `flash_fulfill` against a buffered intent collects the writer's rent on close. The reward path is unaffected, but the writer loses the rent (~0.04 SOL per buffer) every time someone other than them fulfills the intent, which breaks the third-party-bootstrap use case the buffer was added to support.

Recommendation: Close the buffer to `writer` instead of `payer`:

```
if close_flash_fulfill_intent {
    let intent = ctx.accounts.flash_fulfill_intent.as_ref().unwrap();
    let writer = ctx
        .accounts
        .writer
        .as_ref()
        .ok_or(FlashFulfillerError::InvalidWriter)?;
    require_keys_eq!(writer.key(), intent.writer, FlashFulfillerError::InvalidWriter);
    ctx.accounts
        .flash_fulfill_intent
        .close(writer.to_account_info())?;
}
```

This requires adding an optional `writer: Option<UncheckedAccount<'info>>` (mut) to `FlashFulfill` and an `InvalidWriter` error variant.

If the team chooses to keep close-to-payer, document the trade-off in NatSpec on `set_flash_fulfill_intent` and `flash_fulfill` so writers know any third party who consumes their buffer claims the rent, and the buffer should only be used when `writer == payer`.

Eco Foundation: Fixed in PR 50.

Cantina Managed: Fix verified.

3.1.2 Native fee can be over-reported

Severity: Informational

Context: events.rs#L11-L12, flash_fulfill.rs#L198-L210

Summary: The `FlashFulfilled` event includes a `native_fee` field, which according to documentation should represent `reward.native_amount - route.native_amount`. However, in practice, any lamports in the contract get swept and reported as fee.

This means that a solver can inflate their own reported fee arbitrarily, and the reported fee cannot be relied on.

Description: `sweep_leftover_native()` forwards `ctx.accounts.flash_vault.lamports()` -- the entire PDA balance -- to the caller-supplied `claimant`, rather than the precise reward-minus-route delta. `flash_vault` is a System-owned PDA that anyone can send SOL to at any time. Any pre-existing donation is silently picked up by the first subsequent `flash_fulfill()`. The `FlashFulfilled.native_fee` event field inherits this imprecision and over-reports solver profit by the donated amount.

Importantly, a solver can send any amount of lamports of their own to the PDA as part of their transaction, knowing that it will be returned to them at no extra risk or cost.

```rust
let native_fee = sweep_leftover_native(&ctx, flash_vault_seeds)?;

if close_flash_fulfill_intent {
    ctx.accounts
        .flash_fulfill_intent
        .close(ctx.accounts.payer.to_account_info())?;
}

emit_cpi!(FlashFulfilled {
    intent_hash,
    claimant: ctx.accounts.claimant.key(),
    native_fee,
});
```

Impact explanation: Previous audits have made clear that it is fine and intended that a solver can easily sweep any leftover funds, as those would be in the contract due to user error and are fair game, so this is not an economic vulnerability.

Recommendation: Check balance on entry into `flash_fulfill()` and subtract it from the final `native_fee`. If necessary, emit the amount that was swept out of the router, preexisting.

Eco Foundation: Fixed in commit 6289a93e.

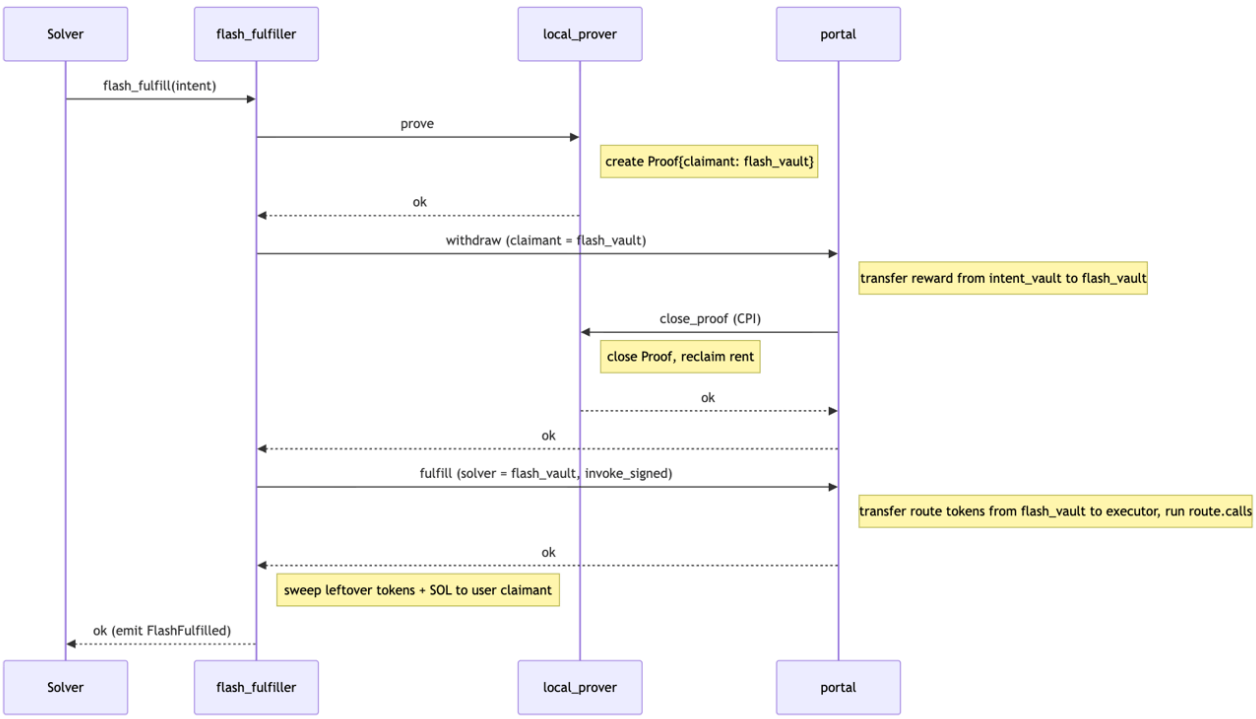
Cantina Managed: Fix verified.

3.1.3 `flash_fulfill` excludes intents whose route or token transfers need any non-trivial nested CPI

Severity: Informational

Context: flash_fulfill.rs#L106

Description: Solana caps the CPI call stack at 4 frames. `flash_fulfill()` adds one frame on top of the standard fulfill path because it sits between the transaction root and `portal::fulfill()`. Every downstream consumer -- token transfers (with or without Token-2022 hooks), executor route calls, and the targets those calls invoke -- has one less frame of budget than it would under a plain `portal::fulfill()`. Intents whose execution requires a CPI chain >1 levels deep at any of those downstream points run via the normal path but revert via `flash_fulfill()`.



The above diagram (courtesy of the Eco team) shows the call sequence. At the right end, there is a "transfer" described, which is its own external call.

This creates two limitations:

1. Transferred Token-2022s cannot have hooks which perform external calls.
2. Route calls have a single frame of call budget: they can perform one external call, whereas they would otherwise be able to perform two.

Impact Explanation: Solvers cannot use `flash_fulfill()` whose route, reward, or route tokens require any CPI chain > 1 level deep at the target program (case 2) or whose Token-2022 hook performs any nested CPI (case 1). No funds are lost. No third party is harmed. The same intent remains fulfillable via standard `portal::fulfill()`.

Likelihood Explanation: The Token-2022 hook subset depends on hook-mint adoption patterns, which today are limited but growing. The route-calls subset depends on exactly the users of the Eco protocol, and there is little reason to expect them to create intents which could not be reasonably fulfilled. However any intent that uses a popular DeFi aggregator already exceeds the budget. It limits usefulness of the `flash_fulfill()` endpoint.

Recommendation: Document prominently in the flash-fulfiller crate-level docstring and README. Make it clear that `flash_fulfill()` works only on a subset of portal-fulfillable intents, and name the conditions: any route call whose target performs 2 levels of nested CPI, or any reward/route token whose Token-2022 transfer hook performs any nested CPI.

Eco Foundation: Acknowledged.

Cantina Managed: Acknowledged.

3.1.4 Undocumented requirements on flash fulfiller transaction sequence

Severity: Informational

Context: *(No context files were provided by the reviewer)*

Description: `flash-fulfiller` installs a 256 KB `BumpAllocator`. The `BumpAllocator` impl bumps down from `start + len` on each allocation. Without an explicit `ComputeBudgetInstruction::request_heap_frame(256 * 1024)` in the same transaction the VM heap defaults to 32 KB. The allocator's first allocation accesses memory beyond that region, which leads to access violation and tx failure.

In the PR description for PR 51, the behavior is documented:

> Test helper prepends `ComputeBudgetInstruction::request_heap_frame(256 * 1024)` to every flash-fulfiller tx - the 256 KB allocator bumps DOWN from start+len, so the first alloc access-violates against the default 32 KB VM heap region without it.

This behavior is indeed in the test. However, it is not documented, apart from being present in the tests and in the PR description. Since this is a hard requirement for certain protocol participants (solvers) to fulfill, it should be more clearly documented.

The test helper enforces this discipline silently. Solvers that don't replicate this behavior produce broken integrations.

Recommendation: Add a paragraph in flash-fulfiller's crate-level `lib.rs` rustdoc explaining that any client building a flash-fulfiller tx MUST prepend `ComputeBudgetInstruction::request_heap_frame(256 * 1024)`.

Eco Foundation: Fixed in commit 11d1d515.

Cantina Managed: Fix verified.

3.1.5 `resolve_intent` can be invoked on a buffer whose address is not derived from the actual intent hash

Severity: Informational

Context: *(No context files were provided by the reviewer)*

Description: `flash_fulfill()`'s `IntentHash(intent_hash)` variant treats the supplied `intent_hash` as a storage index rather than a commitment to the buffer's content. `resolve_intent` only checks that the buffer's address matches `pda(writer.key(), intent_hash)`. It does not verify that `intent_hash == keccak(CHAIN_ID, stored_route.hash(), stored_reward.hash())`. A writer using `append_flash_fulfill_intent_chunk()` can publish a buffer at PDA seed `(writer, H_advertised)` whose stored content actually hashes to `H_real`, with `H_real != H_advertised`. A solver who consumes the buffer thinking it represents `H_advertised` actually fulfills the real intent at `H_real`, which may have different (and possibly hostile) economics, route calls, or reward.

This differs from `set_flash_fulfill_intent()`, which computes `intent_hash = keccak(CHAIN_ID, route.hash(), reward.hash())` from the typed args and derives `pda(writer, intent_hash)`. The `flash_fulfill_intent()` account passed in must match this address.

```
require!(
    ctx.accounts.flash_fulfill_intent.key() == expected_pda,
    FlashFulfillerError::InvalidFlashFulfillIntentAccount
);
```

Solvers are responsible for their own security and are generally considered competent, and are assumed to run simulations. However, upholding the invariant that a consumed buffer address is consistently hashed from the intent it represents, and checking it is cheap.

Recommendation: Add the following check in `programs/flash-fulfiller/src/instructions/flash_fulfill.rs`, in `resolve_intent()`:

```
require!(
    intent_hash == types::intent_hash(
        CHAIN_ID,
        &intent.route.hash(),
        &intent.reward.hash(),
    ),
    FlashFulfillerError::InvalidFlashFulfillIntentAccount
);
```

Eco Foundation: Fixed in commit 2ad2df42.

Cantina Managed: Fix verified.